JARVIS Core Ecosystem: ADHS-Family-Plugin

Architektur- und Implementierungsplan für ein neurodiverses, lokales Familien-Assistenzsystem

Dieses Dokument beschreibt die Konzeption und den Strukturplan für das **ADHS-Family-Plugin** innerhalb des lokalen KI-Ökosystems `JARVIS`. Das System ist speziell für neurodiverse Haushalte ausgelegt und berücksichtigt sowohl betroffene Kinder als auch betroffene Elternteile (Szenario: ein oder beide Partner mit ADHS). Die Oberfläche wird als progressive Web-App (PWA) realisiert, um maximale Barrierefreiheit ohne native App-Zwänge auf Smartphones bereitzustellen.

1. Zielgruppen-Architektur: Multi-User Neurodiversität

Das Plugin unterscheidet nicht starr zwischen „gesunden Verwaltern“ und „zu strukturierenden Kindern“. Da exekutive Dysfunktionen bei erwachsenen ADHS-Betroffenen denselben Mechanismen unterliegen, basiert das System auf einem gleichwertigen, aber rollenbasierten Multi-User-Ansatz:

- **Rolle „Kind/Jugendlicher“:** Fokus auf spielerische Führung, Gamification (Tamagotchi), visuelle Timer und schrittweise Mikro-Tasks zur Vermeidung von lähmender Reizüberflutung.
- **Rolle „Erwachsener / Elternteil (ADHS)“:** Fokus auf Entlastung des Mental Load, Reduzierung von administrativem Setup, Reizminimierung, Entlastung des Arbeitsgedächtnisses durch das lokale RAG-Gedächtnis und Vermeidung von „Schuldgefühlen“ bei nicht geschafften Tasks.
- **Rolle „Erwachsener (Neurotypisch)“:** Optionale Rolle zur reinen Einsicht des synchronisierten Familienstatus, ohne neurodiverse Assistenz-Features im eigenen Profil.

2. Das Gamification- & Tamagotchi-Konzept

Um die typische „ADHS-Langeweile“ nach wenigen Tagen der Nutzung zu brechen, implementiert das System ein kooperatives Familiensystem, das direkt auf das Dopamin-Management einzahlt.

Das Prinzip des „Familien-Tandems“ (Kooperatives Tamagotchi)

Anstatt dass jedes Familienmitglied egoistisch ein eigenes Haustier pflegt, existiert ein **zentrales Familien-Tamagotchi (oder ein ausbaufähiges Familien-Hauptquartier)**. Jedes Mitglied zahlt durch das Erledigen seiner individuellen, altersgerechten Routinen in den globalen Dopamin- und Ressourcenpool ein.

- **Der Dopamin-Effekt:** Schafft das Kind seine Morgenroutine, erhält das Tamagotchi Energie. Vergisst der Vater seine Rechnungen nicht, schaltet er Zubehör frei.
- **Schutz vor Frust:** Das Haustier kann *niemals sterben* (dies würde ADHS-typische Scham- und Versagensängste triggern). Bei Inaktivität wird es lediglich „müde“ oder „geht auf Entdeckungsreise“ und bringt beim Wiederaufleben kleine Geschenke mit.

3. Technische Modulübersicht (JARVIS-Plugin)

Das Plugin nutzt die Kernkompetenzen des lokalen JARVIS-Backends (LLM, Embeddings, lokale Speicherung) und stellt diese über eine REST- bzw. WebSocket-API für die mobile Weboberfläche bereit.

Sub-Modul	Funktion im Backend	Smartphone-PWA Schnittstelle / UI
1. Micro-Task Parser	Nutzung des lokalen LLMs zur Zerlegung komplexer Befehle in atomare Einzelschritte.	**„1-Klick-Routine“:** Ein fetter Button bspw. für „Morgenroutine Lukas“ oder „Steuererklärung Papa“. Zeigt immer nur *einen* Teilschritt groß im Screen.
2. Neuro-Timer	Zustandsmaschine (State Machine) für adaptive Intervalle und intelligentes Nudging.	**Visuelle Schrumpfscheibe:** Keine Zahlen, sondern ein farbiger Kreis, der abnimmt. Bei Ablauf erfolgt sanfte Vibration und Audio-Nachfrage.
3. Kooperatives Tamagotchi	Gamification-Engine. Berechnet XP, Levels, Statusänderungen und Belohnungs-Freischaltungen.	**Zentraler Dashboard-Tab:** Animierte SVG/Canvas-Grafik des Familien-Haustiers. Interaktive Animationen bei Task-Erfüllung.
4. Lokales RAG-Tagebuch	Speichert formlose Logs (Text/Sprache) in lokaler Chroma/FAISS Vektordatenbank.	**Sprachnotiz-Button:** Schnelles Einsprechen von Verhaltensmustern oder Medikations-Feedback („Rebound um 17 Uhr“).

4. PWA Smartphone-Spezifikationen (Ohne App-Store-Zwang)

Da das System als reine Webapplikation ohne native Installation laufen muss, werden moderne Web-APIs genutzt, um App-Feeling zu erzeugen:

A. Das Objekt-Permanenz-Problem lösen (Web-Plattform)

- **Service Worker & Push API:** JARVIS sendet Push-Nachrichten über lokale Web-Sockets oder Web-Push-Dienste direkt auf das Smartphone, selbst wenn der Browser geschlossen ist (unter iOS ab Version 16.4 nativ unterstützt).
- **Homescreen-Installation (Manifest.json):** Über den „Zum Startbildschirm hinzufügen“-Dialog wird die Webapp im Standalone-Modus geöffnet – ohne Adresszeile, exakt wie eine native Anwendung.
- **Web Vibration API:** Haptisches Feedback bei der Bestätigung von Aufgaben zur Erhöhung des physischen Belohnungsgefühls.

B. UI/UX Richtlinien für ADHS-Gehirne (Mobile First)

- **Dark-Mode standardmäßig aktiviert:** Reduziert die visuelle Erschöpfung. Muted-Colors statt greller Neonfarben, um Überstimulation zu vermeiden.
- **Zero-Text-Entry-Philosophie:** Eingaben für Routinen erfolgen über große Buttons, Icons oder Sprache. Textfelder werden im Alltag komplett vermieden.
- **Unendlicher Vergebungsmodus:** Verpasste Aufgaben erzeugen keine roten Fehlermeldungen oder Minus-Punkte. Sie verschwinden einfach stillschweigend oder verschieben sich elastisch auf den Folgetag.

5. Implementierungs-Roadmap für die JARVIS-Integration

Phase 1: Backend API & Datenmodell (JARVIS Plugin Core)

Erweiterung der JARVIS-Plugin-Struktur um die Endpunkte `/api/adhs/tasks`, `/api/adhs/gamification` und `/api/adhs/rag-log`. Speicherung der Benutzer-Profile und des Tamagotchi-Status in einer schlanken, lokalen SQLite-Instanz.

Phase 2: Der PWA-Prototyp (Frontend)

Erstellung des Web-Frontends (z.B. mittels Vue.js oder Vanilla JS mit TailwindCSS), optimiert auf mobile Viewports (HTML5 Canvas für das Tamagotchi, CSS-Animationen für die Timer). Integration des `manifest.json` für die Standalone-Fähigkeit.

Phase 3: Der LLM-Promp-Engine Feinschliff

Konfiguration der lokalen System-Prompts, um eine strukturierte und empathische Kommunikation zu garantieren. Einbindung des lokalen Vektorspeichers zur semantischen Analyse eingegangener Elterntagebücher.